

LABORATORY OBSERVATION / RECORD

Field	Details
Course	Full Stack Web Development
Course Code	23CM4121
Branch	CSM
Section	B
Task No.	6 (PBL - Part 2)
Student Name	M. Niharika
Roll No.	A24126552097

This record contains four experiments carried out under Task 6. Each experiment is documented with its aim, objective, theory, algorithm, program, code explanation, and result. Since the questions for this task are conceptual in nature, the theory sections have been elaborated in greater depth to cover the underlying concepts thoroughly.

Q. No.	Question	CO
5	Differentiate between JavaScript functions and classes. Explain how a class can be used to create multiple objects with common properties and methods.	CO-1
6	What are Node.js modules? Explain the purpose of the os, path, and fs modules with suitable examples.	CO-2
7	Explain NPM and package.json in Node.js. Describe the steps involved in installing an external package and using it in a Node.js application.	CO-2
8	What is middleware in Express.js? Explain how middleware works in the request-response cycle. Develop an example of middleware used for logging incoming requests.	CO-2

EXPERIMENT 5

1. TITLE

Differentiating JavaScript Functions and Classes, and Creating Multiple Objects Using a Class.

2. AIM

To differentiate between JavaScript functions and classes, and to demonstrate how a class can be used as a blueprint to create multiple objects that share common properties and methods.

3. OBJECTIVE

- To understand what a function is in JavaScript and the different ways a function can be written and used.
- To understand what a class is in JavaScript and how it differs from a plain function, even though a class is itself a special kind of function under the hood.
- To identify the syntactic and behavioural differences between functions and classes, such as hoisting, invocation, and the use of the new keyword.
- To learn how a constructor initialises the properties of an object at the time it is created.
- To learn how methods defined inside a class are shared by every object created from that class through the prototype, rather than being copied into each object.
- To create multiple independent objects from a single class and to display the properties of each object separately.

4. THEORY

A function in JavaScript is a reusable block of code designed to perform a particular task. It can be defined as a function declaration, a function expression, or an arrow function, and it can optionally accept parameters and return a value. A function on its own does not describe the structure of an object; it simply performs an action whenever it is called, and the same function can be called any number of times with different arguments.

A class, introduced in ECMAScript 2015 (ES6), is a template or blueprint used to create objects that share the same structure and behaviour. A class groups together the properties (data) and the methods (behaviour) that describe a category of objects, such as a Student, a Car or a BankAccount. Internally, a JavaScript class is still built on top of the language's existing prototype-based inheritance model — a class is technically a special kind of function — but it provides a cleaner, more readable syntax for defining constructor logic and methods compared to writing prototype functions manually.

4.1 Key Differences Between Functions and Classes

Aspect	Function	Class
Purpose	Performs a task or computation; may or may not return a value.	Acts as a blueprint that describes the properties and methods of a category of objects.
Declaration keyword	function keyword, or an arrow function using <code>=></code> .	class keyword, followed by a constructor and method definitions.

Aspect	Function	Class
Hoisting	Function declarations are hoisted to the top of their scope and can be called before they appear in the code.	Classes are hoisted, but they remain in a 'temporal dead zone' and cannot be used before the line where they are declared.
Invocation	Can be invoked directly, e.g. <code>greet()</code> or <code>greet("Niharika")</code> .	Cannot be invoked directly; it must be instantiated using the <code>new</code> keyword, e.g. <code>new Student(...)</code> .
Calling without 'new'	A normal function called without <code>new</code> simply executes its body.	Calling a class without <code>new</code> throws a <code>TypeError: Class constructor cannot be invoked without 'new'</code> .
Strict mode	Runs in whatever mode the surrounding code is in.	The body of a class always executes in strict mode automatically, even if <code>'use strict'</code> is not written.
Method sharing	A plain function has no built-in concept of shared instance methods; this has to be arranged manually through the prototype.	Methods written inside a class body are automatically placed on the class's prototype, so every instance shares one copy of each method instead of duplicating it.
Encapsulation	Groups only a single operation.	Groups related data (via the constructor) and behaviour (via methods) together in one named unit, which is closer to true object-oriented design.

4.2 How a Class Creates Multiple Objects with Common Properties and Methods

A class defines two things: a constructor and one or more methods. The constructor is a special function, always named `constructor()`, that runs automatically every time the `new` keyword is used with the class. Its job is to receive the values passed in and assign them to the newly created object using the `this` keyword, so `this.name = name` stores the value received in the parameter `name` onto the property `name` of the object currently being built.

Every object created from the same class receives its own independent copy of the properties assigned inside the constructor, because each call to `new Student(...)` creates a brand-new object in memory and runs the constructor against that specific object. This is why `student1.name` and `student2.name` can hold completely different values even though both objects came from exactly the same class.

Methods, on the other hand, are not copied into every object individually. When a method such as `getDetails()` is written inside the class body, JavaScript places a single copy of that method on `Student.prototype`. Every object created with `new Student(...)` is internally linked to this shared prototype, so calling `student1.getDetails()` and `student2.getDetails()` both run the exact same function, just with `'this'` pointing to the object on which the method was called. This is what allows many different objects to share common behaviour without wasting memory on duplicate copies of the same method.

In short, a class is used to create multiple objects with common properties and methods in the following way: the properties come from the constructor and are unique per object, while the methods come from the class body and are shared by all objects through the prototype. The keyword `this` inside a method always refers to whichever object

the method was called on, which is what makes the same shared method produce a different, object-specific result each time.

4.3 Key Concepts Used

- `class Student { ... }` — defines a class named `Student` that acts as a blueprint for student objects.
- `constructor(name, rollNumber, course)` — a special method that runs once, automatically, whenever a new object is created from the class.
- `this.name = name` — inside the constructor, `this` refers to the object currently being constructed, and the assignment stores the parameter's value on that object.
- `new Student(...)` — the `new` operator creates a new empty object, links it to `Student.prototype`, runs the constructor with `this` bound to that object, and returns the finished object.
- `getDetails()` — an instance method defined in the class body; it is stored once on the prototype and is available to every object created from the class.
- Template literals (``...${this.name}...``) — used inside `getDetails()` to build a formatted multi-line string from the object's own properties.
- `document.getElementById("output").innerHTML` — used to write the combined details of all three objects onto the web page.

5. ALGORITHM / PROCEDURE

- Create two files, `index.html` and `script.js`, and link `script.js` at the end of the body of `index.html`.
- In `index.html`, add a heading and an empty `<div id="output"></div>` that will hold the generated student details.
- In `script.js`, define a class named `Student` using the `class` keyword.
- Inside the class, write a `constructor(name, rollNumber, course)` that assigns each parameter to a property of the object using `this.name`, `this.rollNumber` and `this.course`.
- Define a method `getDetails()` inside the class that returns a formatted string containing the object's own name, `rollNumber` and `course`, built using a template literal.
- Create three separate objects — `student1`, `student2` and `student3` — using the `new` keyword with different arguments for each.
- Call `getDetails()` on each of the three objects and concatenate the results into a single HTML string.
- Write the combined string into the output `<div>` using `innerHTML`, and save both files.
- Open `index.html` in a browser and confirm that all three students are displayed with their own distinct details, even though they were built from the same class and the same `getDetails()` method.

6. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JavaScript Classes</title>
</head>
<body>
  <h1>Student Details</h1>
  <div id="output"></div>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Create a Student class
class Student {

  // Constructor initializes object properties
  constructor(name, rollNumber, course) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.course = course;
  }

  // Method to display student details
  getDetails() {
    return `
      Name: ${this.name}<br>
      Roll Number: ${this.rollNumber}<br>
      Course: ${this.course}<br>
    `;
  }
}

// Create multiple objects from the same class
const student1 = new Student(
  "Niharika",
  101,
  "CSE"
);

const student2 = new Student(
  "Sneha",
  102,
  "ECE"
);

const student3 = new Student(
  "Priya",
  103,
  "AI & ML"
);

// Display objects
document.getElementById("output").innerHTML =
```

```

"<h3>Student 1</h3>" +
student1.getDetails() +
"<hr>" +

"<h3>Student 2</h3>" +
student2.getDetails() +
"<hr>" +

"<h3>Student 3</h3>" +
student3.getDetails();

```

7. CODE EXPLANATION

Code	Explanation
class Student { ... }	Declares a class named Student that acts as a reusable blueprint for creating student objects.
constructor(name, rollNumber, course)	Runs automatically each time a new object is created; receives the three arguments and stores them on the new object.
this.name = name	Assigns the value received in the parameter to a property on the object currently being constructed.
getDetails()	An instance method placed once on Student.prototype; shared by every object created from the class.
new Student("Niharika", 101, "CSE")	Creates a brand-new object, links it to the class's prototype, and runs the constructor with this bound to that object.
Template literal in getDetails()	Builds a multi-line HTML-formatted string using the object's own name, rollNumber and course.
student1.getDetails(), student2.getDetails()	Each call runs the same shared method, but this refers to a different object each time, producing different output.
innerHTML	Writes the combined HTML string of all three students' details onto the page through the DOM.

8. OUTPUT

On opening index.html in a browser, the page displays a heading "Student Details" followed by the details of Student 1 (Niharika, Roll Number 101, CSE), Student 2 (Sneha, Roll Number 102, ECE) and Student 3 (Priya, Roll Number 103, AI & ML), each separated by a horizontal rule. Even though all three objects were created from the same Student class and call the exact same getDetails() method, each block of output is distinct because this inside the method refers to the specific object it was called on.

9. RESULT

Thus, the difference between JavaScript functions and classes was studied, and a class was successfully used to create multiple objects with common properties and methods. It was observed that the constructor assigns independent property values to each object, while methods defined in the class body are shared by all objects through the prototype, allowing the same code to describe many different, independently behaving objects.

EXPERIMENT 6

1. TITLE

Node.js Modules — Understanding and Using the os, path and fs Built-in Modules.

2. AIM

To explain the concept of modules in Node.js and to demonstrate the purpose of the os, path and fs built-in modules with suitable examples.

3. OBJECTIVE

- To understand what a module is in Node.js and why code is organised into modules rather than written in a single file.
- To distinguish between core (built-in) modules, local modules and third-party modules.
- To import a built-in module using `require()` and understand the CommonJS module system.
- To retrieve operating-system-level information using the os module.
- To construct and inspect file paths in an operating-system-independent way using the path module.
- To create, read and append to a file using the fs module, and to understand the difference between synchronous and asynchronous file operations.

4. THEORY

Node.js follows a modular architecture: instead of writing an entire application in one large file, related pieces of functionality are grouped into separate files called modules, and modules are loaded into other files only when they are actually needed. This keeps the code organised, makes it reusable across different parts of an application, and avoids polluting the global scope, because everything declared inside a module stays private to that module unless it is explicitly exported.

Node.js recognises three broad categories of modules. Core modules (also called built-in modules) ship with Node.js itself and provide access to system-level functionality such as the file system, networking and the operating system; they can be loaded directly by name, for example `require("fs")`, without installing anything. Local modules are files written by the developer as part of the project and are imported using a relative path, for example `require("./fileOperations")`. Third-party modules are packages published by other developers and installed through NPM into the `node_modules` folder before they can be required by name.

Node.js uses the CommonJS module system by default. Every file is treated as its own module with its own scope, and `require("moduleName")` synchronously loads that module and returns whatever it exposed through `module.exports`. Because `require()` reads the module once and caches the result, requiring the same module a second time anywhere in the application returns the same cached object instead of re-executing the file.

4.1 The os Module

The os module provides information about the underlying operating system and hardware on which the Node.js process is running. It is a read-only, informational module — it does not change anything about the system, it only

reports facts about it, which makes it useful for logging, diagnostics, and writing code that needs to behave slightly differently depending on the platform it runs on.

- `os.platform()` — returns a string identifying the operating system, such as 'win32', 'linux' or 'darwin'.
- `os.arch()` — returns the CPU architecture the Node.js binary was compiled for, such as 'x64' or 'arm64'.
- `os.cpus()` — returns an array with one entry per logical CPU core, including its model and clock speed; `os.cpus().length` gives the number of cores.
- `os.homedir()` — returns the current user's home directory as an absolute path.
- `os.totalmem()` / `os.freemem()` — return the total and currently free system memory in bytes, useful for basic resource monitoring.

4.2 The path Module

The path module supplies utilities for working with file and directory paths in a way that is independent of the operating system. Windows uses a backslash (\) as its path separator while Linux and macOS use a forward slash (/); building paths by manually concatenating strings with a hard-coded separator therefore breaks as soon as the code is run on a different platform. The path module hides this difference by always producing a correctly formatted path for whichever operating system the code is currently running on.

- `path.join(...segments)` — joins multiple path segments together using the correct separator for the current platform and normalises the result, e.g. removing redundant slashes.
- `path.basename(fullPath)` — returns just the file name portion of a path, without any of the preceding directories.
- `path.dirname(fullPath)` — returns the directory portion of a path, with the file name removed.
- `path.extname(fullPath)` — returns the file extension, including the leading dot, such as '.txt'.
- `__dirname` — a special variable, available in every CommonJS module, that always holds the absolute path of the directory containing the current file; it is commonly combined with `path.join()` to build reliable absolute paths regardless of where the script is run from.

4.3 The fs Module

The fs (File System) module gives a Node.js program direct read and write access to files and directories on disk, something a browser-based JavaScript program is not normally permitted to do for security reasons. Almost every method in the fs module is available in two forms: an asynchronous, non-blocking version that accepts a callback (or returns a Promise when used with `fs.promises`), and a synchronous, blocking version whose name ends in `Sync`, such as `fs.writeFileSync()`.

The synchronous versions used in this experiment — `writeFileSync()`, `readFileSync()` and `appendFileSync()` — are simpler to read because they run one after another in plain top-to-bottom order and their result can be used immediately on the next line. Their disadvantage is that each call blocks the entire Node.js process until the disk operation finishes, so in a server application, where many requests are being handled at once, the asynchronous versions (or `fs.promises` with `async/await`) are generally preferred so that one slow file operation cannot delay every other request.

- `fs.writeFileSync(filename, content)` — creates a new file with the given content, or completely overwrites the file if it already exists.
- `fs.readFileSync(filename, "utf8")` — reads the entire contents of a file and returns it as a readable text string; without the "utf8" encoding argument it would return a raw Buffer of bytes instead.
- `fs.appendFileSync(filename, text)` — adds the given text to the end of an existing file without deleting or overwriting whatever content is already there.
- Passing an encoding such as "utf8" tells Node.js how to convert the raw bytes on disk into a JavaScript string, and is what allows `console.log()` to print human-readable text instead of a Buffer object.

4.4 Summary of Purpose

Module	Primary Purpose	Typical Use Case
os	Reports information about the operating system and hardware.	Logging the platform, checking available memory, tailoring behaviour to the OS.
path	Builds and inspects file paths in a cross-platform, error-free way.	Combining <code>__dirname</code> with a file name to build a reliable absolute path.
fs	Reads from and writes to files and directories on disk.	Creating log files, saving user data, reading configuration files.

5. ALGORITHM / PROCEDURE

- Create a new file, e.g. `modules.js`, and import the three built-in modules using `require("os")`, `require("path")` and `require("fs")`.
- Under the OS module section, call `os.platform()`, `os.arch()`, `os.cpus().length` and `os.homedir()`, and print each result using `console.log()`.
- Under the PATH module section, build a sample file path using `path.join(__dirname, "data", "student.txt")` and print the joined path.
- From that same path, extract and print the file name with `path.basename()`, the directory with `path.dirname()`, and the extension with `path.extname()`.
- Under the FS module section, define a file name and some text content, then create the file using `fs.writeFileSync()`.
- Read the file back with `fs.readFileSync(fileName, "utf8")` and print its contents to confirm the write succeeded.
- Append additional text to the same file using `fs.appendFileSync()`, then read the file a final time and print the combined contents.
- Run the script with `node modules.js` and verify that the OS details, the path details and the final file contents all appear correctly in the terminal, and that the file was actually created in the project folder.

6. PROGRAM

`modules.js`

```
// Import built-in modules
```

```

const os = require("os");
const path = require("path");
const fs = require("fs");

// -----
// 1. OS MODULE
// -----

console.log("==== OS MODULE ====");

console.log("Operating System:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Number of CPUs:", os.cpus().length);
console.log("Home Directory:", os.homedir());

// -----
// 2. PATH MODULE
// -----

console.log("\n==== PATH MODULE ====");

const filePath = path.join(
  __dirname,
  "data",
  "student.txt"
);

console.log("Complete Path:", filePath);
console.log("File Name:", path.basename(filePath));
console.log("Directory:", path.dirname(filePath));
console.log("Extension:", path.extname(filePath));

// -----
// 3. FS MODULE
// -----

console.log("\n==== FS MODULE ====");

const fileName = "example.txt";
const content = "Hello from Node.js File System module!";

// Create/write a file
fs.writeFileSync(fileName, content);

console.log("File created successfully.");

// Read the file
const fileData = fs.readFileSync(fileName, "utf8");

console.log("File contents:");

```

```

console.log(fileData);

// Append content
fs.appendFileSync(
  fileName,
  "\nThis is additional content."
);

console.log("Content appended successfully.");

// Read final content
const finalData = fs.readFileSync(
  fileName,
  "utf8"
);

console.log("\nFinal file contents:");
console.log(finalData);

```

7. CODE EXPLANATION

Code	Explanation
<code>require("os") / require("path") / require("fs")</code>	Loads the three built-in Node.js modules using the CommonJS <code>require()</code> function; no installation is needed since these ship with Node.js.
<code>os.platform() / os.arch()</code>	Report the operating system name and CPU architecture the script is currently running on.
<code>os.cpus().length</code>	Counts the number of logical CPU cores available on the machine.
<code>os.homedir()</code>	Returns the absolute path of the current user's home directory.
<code>path.join(__dirname, "data", "student.txt")</code>	Combines the current directory with two more segments into one correctly formatted path for the current OS.
<code>path.basename() / path.dirname() / path.extname()</code>	Extract the file name, the directory, and the extension respectively from the joined path.
<code>fs.writeFileSync(fileName, content)</code>	Creates <code>example.txt</code> (or overwrites it if it already exists) with the given text.
<code>fs.readFileSync(fileName, "utf8")</code>	Reads <code>example.txt</code> and returns its contents as a plain text string.
<code>fs.appendFileSync(fileName, "...")</code>	Adds a new line of text to the end of <code>example.txt</code> without removing the existing content.

8. OUTPUT

Running `node modules.js` prints the OS section (platform, architecture, CPU count and home directory), followed by the PATH section showing the joined path together with its file name, directory and extension, and finally the FS section confirming that the file was created. The file contents printed after the write show only the original line, while the final read — printed after the append — shows both lines together, exactly as stored in `example.txt`:

```

Hello from Node.js File System module!
This is additional content.

```

9. RESULT

Thus, the concept of modules in Node.js was explained, and the `os`, `path` and `fs` built-in modules were demonstrated successfully. The `os` module was used to read operating-system and hardware information, the `path` module was used to build and break down a file path in a platform-independent way, and the `fs` module was used to create, read and append to a file, confirming how modularity allows Node.js to expose powerful system-level capabilities through small, focused, reusable pieces of functionality.

EXPERIMENT 7

1. TITLE

NPM and package.json — Installing and Using an External Package in a Node.js Application.

2. AIM

To explain NPM and the package.json file in Node.js, and to describe and demonstrate the steps involved in installing an external package and using it inside a Node.js application.

3. OBJECTIVE

- To understand what NPM is and the role it plays in the Node.js ecosystem.
- To understand the purpose of package.json as the manifest, or identity card, of a Node.js project.
- To distinguish package.json from package-lock.json and to understand why both files exist.
- To understand semantic versioning and the meaning of the caret (^) symbol used in dependency version ranges.
- To install a third-party package using npm install and observe how it updates the project's files.
- To import and use the installed package inside application code and run the project using an npm script.

4. THEORY

NPM stands for Node Package Manager. It is installed automatically alongside Node.js and serves two purposes: it is a command-line tool used to install, update and remove packages in a project, and it is also the name of the world's largest public registry of JavaScript packages, hosted at registry.npmjs.org, from which those packages are downloaded. Without a package manager such as NPM, a developer would have to manually download, place and keep track of every third-party library a project depends on, which quickly becomes unmanageable as a project grows.

Every Node.js project intended to use NPM has a package.json file at its root. This file is a JSON document that acts as the manifest of the project: it records the project's name, version and description, the command used to start it, and — most importantly — the list of external packages (dependencies) the project needs in order to run. package.json is created either manually or, more commonly, by running npm init (which asks a series of questions) or npm init -y (which accepts sensible defaults for all of them without asking).

4.1 Structure of package.json

- name / version — the identifying name and semantic version of the project itself.
- description — a short, human-readable summary of what the project does.
- main — the entry-point file that is loaded when another module requires this package, or that npm start commonly runs.
- scripts — a set of named shortcuts for shell commands; for example "start": "node app.js" allows the longer command to be run simply as npm start.
- dependencies — packages the application needs at runtime in order to work at all, such as express or chalk.

- `devDependencies` — packages needed only during development, such as testing or bundling tools, but not needed once the application is deployed and running.
- `license` / `author` / `keywords` — metadata about the project that does not affect how it runs.
- `type` — set to `"module"` to use ES module `import/export` syntax, or left as the default `"commonjs"` to use `require()/module.exports`.

4.2 package.json vs package-lock.json

`package.json` lists dependencies using version ranges rather than exact versions — for example `"chalk": "^6.0.0"` means 'any 6.x.x version that is compatible with 6.0.0', following semantic versioning rules where the caret (^) allows minor and patch updates but blocks a jump to a new major version. This flexibility is convenient during development, but it means that two different installs of the same `package.json`, performed weeks apart, could theoretically resolve to two slightly different actual versions.

`package-lock.json` solves this by recording the exact version, download URL and integrity checksum of every package that was actually installed, including the full tree of sub-dependencies those packages themselves rely on. Whenever `npm install` is run again, NPM reads `package-lock.json` and installs exactly those pinned versions, guaranteeing that every developer on a team, and every deployment of the application, ends up with an identical set of installed packages. `package-lock.json` is generated and updated automatically by NPM and is not normally edited by hand.

4.3 Steps to Install and Use an External Package

- Initialise the project (if not already done) by running `npm init -y` inside the project folder, which creates `package.json` with default values.
- Install the desired package from the NPM registry by running `npm install <package-name>` — for example, `npm install chalk`. This downloads the package, and any packages it depends on, into a `node_modules` folder.
- NPM automatically adds an entry for the package to the `dependencies` section of `package.json`, recording an acceptable version range.
- NPM also creates or updates `package-lock.json` with the exact version and integrity hash of everything that was installed, so future installs are reproducible.
- Import the package into application code — using `import chalk from "chalk"` (ES module syntax, when `"type": "module"` is set in `package.json`) or `const chalk = require("chalk")` (CommonJS syntax).
- Use the functionality the package provides — in this example, chalk's colour-formatting functions such as `chalk.green(text)`.
- Run the application, either directly with `node app.js` or through a script defined in `package.json`, such as `npm start`.

4.4 Key Concepts Used

- `npm install <package>` — downloads a package from the registry and adds it as a project dependency.

- `node_modules` — the folder where all installed packages (and their own dependencies) physically live; it is typically excluded from version control because it can always be regenerated from `package.json` and `package-lock.json`.
- Semantic versioning (MAJOR.MINOR.PATCH) — a version numbering convention where the major number signals breaking changes, the minor number signals new backward-compatible features, and the patch number signals bug fixes.
- The caret (^) in "`^6.0.0`" — permits automatic updates to any version from 6.0.0 up to, but not including, 7.0.0.
- npm scripts — named commands under the `scripts` field of `package.json`, run using `npm run <script-name>`, or directly as `npm start` / `npm test` for those two reserved names.
- `import` / `require` — the two ways a package's exported functionality is brought into a JavaScript file, depending on whether the project uses ES modules or CommonJS.

5. ALGORITHM / PROCEDURE

- Create a new project folder and initialise it by running `npm init -y`, which generates a default `package.json`.
- Install the chalk package by running `npm install chalk` in the terminal, and confirm that a `node_modules` folder and a `package-lock.json` file are created.
- Open `package.json` and confirm that chalk now appears under dependencies with a version range such as "`^6.0.0`".
- Create an entry-point file, `app.js`, and import chalk at the top using `import chalk from "chalk"` (setting "`type`": "`module`" in `package.json` so ES module syntax is understood).
- Use chalk's colour functions — `chalk.green()`, `chalk.blue()` and `chalk.yellow()` — inside `console.log()` statements to print coloured messages.
- Add a "`start`": "`node app.js`" script to `package.json` so the application can be launched with `npm start`.
- Run the application with `node app.js` (or `npm start`) and verify that the messages are printed in the terminal in the corresponding colours.

6. PROGRAM

app.js

```
import chalk from "chalk";

console.log(chalk.green("NPM package installed successfully!"));
console.log(chalk.blue("This message is displayed using Chalk."));
console.log(chalk.yellow("Node.js makes package management easy."));
```

package.json

```
{
  "name": "07-npm_demo",
  "version": "1.0.0",
  "description": "",
  "type": "module",
```

```

"main": "app.js",
"scripts": {
  "start": "node app.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
  "chalk": "^6.0.0"
}
}

```

package-lock.json (excerpt)

```

{
  "name": "07-npm_demo",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "07-npm_demo",
      "version": "1.0.0",
      "license": "ISC",
      "dependencies": {
        "chalk": "^6.0.0"
      }
    },
    "node_modules/chalk": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/chalk/-/chalk-6.0.0.tgz",
      "license": "MIT",
      "engines": { "node": ">=22" }
    }
  }
}

```

7. CODE EXPLANATION

Code	Explanation
import chalk from "chalk"	Imports the default export of the installed chalk package so its colour functions can be used in this file.
chalk.green("...")	Wraps the given text with terminal colour codes so it prints in green when logged.
chalk.blue("...") / chalk.yellow("...")	Format text in blue and yellow respectively, demonstrating that many colours can be chained or used independently.
"dependencies": { "chalk": "^6.0.0" }	The entry NPM automatically added to package.json after npm install chalk was run.
"type": "module"	Tells Node.js to interpret .js files using ES module (import/export) syntax rather than CommonJS.
"scripts": { "start": "node app.js" }	Defines a shortcut so the application can be launched with npm start instead of typing node app.js.

Code	Explanation
package-lock.json → resolved / version	Records the exact tarball URL and version of chalk that was installed, ensuring reproducible installs in the future.

8. OUTPUT

Running `node app.js` (or `npm start`) prints three lines in the terminal, each rendered in a different colour by the chalk package: "NPM package installed successfully!" in green, "This message is displayed using Chalk." in blue, and "Node.js makes package management easy." in yellow, confirming that the externally installed package was correctly imported and used.

9. RESULT

Thus, NPM and `package.json` were explained, and the steps involved in installing and using an external package in a Node.js application were successfully demonstrated. The chalk package was installed using `npm install`, which updated `package.json`, created `package-lock.json`, and downloaded the package into `node_modules`; it was then imported and used to print colour-formatted messages, confirming how NPM manages a project's dependencies from installation through to use in code.

EXPERIMENT 8

1. TITLE

Middleware in Express.js — Understanding the Request-Response Cycle and Building a Logging Middleware.

2. AIM

To explain what middleware is in Express.js, to explain how middleware works within the request-response cycle, and to develop an example of middleware used for logging incoming requests.

3. OBJECTIVE

- To understand what middleware means in the context of an Express.js application.
- To understand the request-response cycle and the exact point at which middleware functions execute within it.
- To learn the standard middleware function signature (req, res, next) and the role played by each of its three parameters.
- To understand the purpose of the next() function and what happens if it is not called.
- To distinguish between the different categories of middleware available in Express.
- To develop a custom logging middleware that records the method, URL and timestamp of every incoming request.

4. THEORY

Middleware in Express.js refers to functions that sit in between the moment a request arrives at the server and the moment a final response is sent back to the client. Every middleware function has access to the request object (req), the response object (res), and a third special function conventionally named next. Middleware can inspect or modify the request and response objects, run any code such as logging or authentication checks, and then either end the request-response cycle itself by sending a response, or pass control forward to the next function in the chain by calling next().

Express applications are, at their core, built almost entirely out of middleware. Even the route handlers written with app.get() or app.post() are themselves middleware functions with the specific job of producing the final response; the only real difference between a 'route handler' and 'general middleware' is where in the chain it sits and whether it usually calls next() or ends the cycle by sending a response.

4.1 The Request-Response Cycle and Where Middleware Fits

When a client sends an HTTP request to an Express server, the request does not go directly to a single handler. Instead, Express passes it through a pipeline of middleware functions in exactly the order they were registered with app.use() or app.get()/app.post() and so on. Each function in this pipeline receives the same req and res objects, so changes one middleware makes — for example, attaching data to req — are visible to every middleware that runs after it.

A middleware function that finishes its work calls `next()`, which hands control to the next matching function in the pipeline; Express does not move on by itself. If a middleware neither sends a response nor calls `next()`, the request simply hangs and the client's browser or tool will wait indefinitely, which is one of the most common beginner mistakes when writing custom middleware. Conversely, if a middleware calls `res.send()` or `res.json()`, the cycle is considered complete for that request, and calling `next()` afterwards (or sending a second response) will cause an error.

Because middleware runs strictly in registration order, placement in the file matters a great deal. Middleware meant to apply to every request — such as request logging, authentication, or JSON body parsing — is registered with `app.use()` before the specific routes, so that it runs first for every incoming request. A catch-all handler intended to respond to unmatched routes with a 404 message, on the other hand, must be registered after all real routes, since Express only reaches it if nothing earlier in the pipeline has already sent a response.

4.2 Categories of Middleware in Express

Category	Description
Application-level	Bound to the app object with <code>app.use()</code> or <code>app.METHOD()</code> ; runs for every request, or only for requests matching a specific path/method.
Router-level	Same idea as application-level middleware, but bound to an <code>express.Router()</code> instance instead of the whole app, allowing it to apply only to a subset of routes.
Built-in	Middleware shipped with Express itself, such as <code>express.json()</code> (parses a JSON request body) and <code>express.static()</code> (serves static files).
Third-party	Middleware published as separate NPM packages, such as <code>cors</code> (handles cross-origin requests) or <code>morgan</code> (HTTP request logging).
Error-handling	A special middleware with four parameters, (<code>err</code> , <code>req</code> , <code>res</code> , <code>next</code>), registered after all other middleware, used to catch and respond to errors.

4.3 The Logging Middleware Developed in This Experiment

The custom middleware built for this experiment, `logger`, demonstrates the application-level, general-purpose category described above. It is registered once with `app.use(logger)` before any routes are defined, which means Express runs it for every single incoming request, regardless of which path or HTTP method the request uses. Inside the function, it reads `req.method` (e.g. GET, POST) and `req.url` (the path that was requested) to build a single line describing the request, along with the current timestamp obtained from `new Date().toLocaleString()`, and prints that line to the server's console using `console.log()`.

After printing the log line, the middleware calls `next()` to hand control on to whichever route handler matches the request. Because `logger` performs no validation and never sends a response of its own, it must call `next()` unconditionally on every request, or every request to the server would hang forever without ever reaching its actual route handler.

4.4 Key Concepts Used

- function `logger(req, res, next) { ... }` — the standard three-parameter signature every ordinary (non-error) Express middleware function follows.
- `req.method` — the HTTP method of the incoming request, such as GET, POST, PUT or DELETE.

- req.url — the path portion of the URL that was requested, for example "/students".
- new Date().toLocaleString() — generates a human-readable timestamp for the moment the request was received.
- next() — passes control to the next middleware or route handler in the pipeline; omitting this call would leave every request unanswered.
- app.use(logger) — registers the logger middleware at the application level, before the routes, so it runs for every incoming request.
- module.exports = logger — exports the middleware function from its own file (middleware/logger.js) so it can be imported into the main server file.

5. ALGORITHM / PROCEDURE

- Create a project folder, run npm init -y, and install Express with npm install express.
- Create a subfolder middleware and inside it a file logger.js.
- In logger.js, define a function logger(req, res, next) that builds a timestamp with new Date().toLocaleString().
- Inside the function, print a single log line to the console containing the timestamp, req.method and req.url.
- Call next() at the end of the function so the request is allowed to continue to the next handler, and export the function with module.exports.
- Create the main server file, index.js, and import Express and the logger middleware with require().
- Create the Express application object with express() and register the logger middleware globally using app.use(logger), before defining any routes.
- Define GET routes for /, /students and /about, each sending a simple response using res.send() or res.json().
- Start the server with app.listen(PORT, callback) and print a confirmation message once it is listening.
- Run the server with node index.js, then visit each route in a browser and confirm that a corresponding log line — with method, URL and timestamp — appears in the terminal for every request.

6. PROGRAM

middleware/logger.js

```
function logger(req, res, next) {
  const currentTime = new Date().toLocaleString();

  console.log(
    `[${currentTime}] ${req.method} ${req.url}`
  );

  next();
}

module.exports = logger;
```

index.js

```
const express = require("express");

const logger = require("../middleware/logger");

const app = express();

const PORT = 3000;

// Register logging middleware
app.use(logger);

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to the Express.js application.");
});

// Students route
app.get("/students", (req, res) => {
  res.json([
    {
      id: 1,
      name: "Niharika"
    },
    {
      id: 2,
      name: "Rahul"
    }
  ]);
});

// About route
app.get("/about", (req, res) => {
  res.send("This application demonstrates Express middleware.");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

package.json

```
{
  "name": "08-express_middleware",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
```

```

"license": "ISC",
"type": "commonjs",
"dependencies": {
  "express": "^5.2.1"
}
}

```

7. CODE EXPLANATION

Code	Explanation
<code>function logger(req, res, next)</code>	Defines a middleware function following Express's standard (req, res, next) signature.
<code>new Date().toLocaleString()</code>	Generates a readable timestamp representing the moment the request was received.
<code>console.log(`\${currentTime} \${req.method} \${req.url}`)</code>	Prints one line per request, showing when it arrived, its HTTP method, and the path requested.
<code>next()</code>	Passes control to the next middleware or route handler; without this call the request would never be answered.
<code>app.use(logger)</code>	Registers logger as application-level middleware, placed before the routes so it runs first for every request.
<code>app.get("/", (req, res) => { res.send(...) })</code>	Defines the home route, which runs only after logger has already logged the request.
<code>res.json([...])</code>	Sends the students array as a JSON response, with the Content-Type header set automatically.
<code>module.exports = logger</code>	Exposes the logger function from its own file so index.js can import it with require().

8. OUTPUT

After starting the server with `node index.js`, the terminal first prints "Server running at http://localhost:3000". Visiting the three routes in a browser produces the expected responses — a welcome message at `/`, a JSON array of two students at `/students`, and a description at `/about` — and, because logger runs before every route, each visit also prints a corresponding line to the server console, for example:

```

[9/22/2026, 10:15:03 AM] GET /
[9/22/2026, 10:15:07 AM] GET /students
[9/22/2026, 10:15:11 AM] GET /about

```

9. RESULT

Thus, middleware in Express.js was explained, its role in the request-response cycle was described, and a working logging middleware was successfully developed. The logger middleware, registered with `app.use()` before the routes, was shown to run for every incoming request, print its method, URL and timestamp to the console, and then call `next()` to allow the matching route handler to send the actual response, confirming how middleware forms the backbone of request processing in an Express.js application.